Design and Construction of a Music Sampler

Edoardo Takanen

May 15, 2026

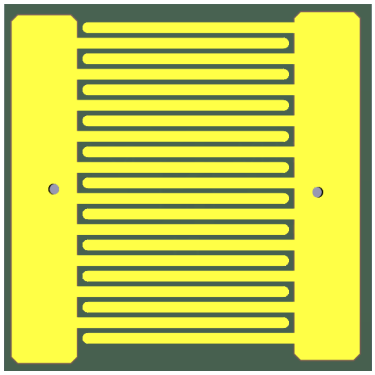
Introduction

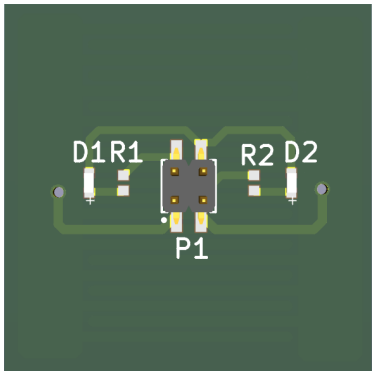
A **sampler** is an electronic musical instrument that **records** and plays back **samples** (digital audio assets). It generally comes with a set of **pressure sensitive pads**, each of them can be configured to reproduce a different sample.
This machine has been frequently used in **hip hop music** for over the past 30 years, with the Akai MPC being the most popular series of Samplers used by producers.
My goal is to **engineer** a custom solution from the ground up. This project was for fun as a hobby but also educational, as it required both designing the hardware and implement a dedicated software–I have learned so many new things and I definitely do **not** consider this project finished, but I will keep on upgrading and perfecting it.
Custom building something always comes with advantages and disadvantages. On one hand, it grants absolute control over the device, being able to implement whatever I want whenever I need some new features has a lot of different outcomes–it makes the device more flexible to future requirements. While on the other hand, there are a lot of challenges to address and options to consider, not to mention that what I am trying to imitate is something so complex that it usually takes a whole team to build–sometimes reinventing the wheel is just stressful!
Moreover, as my last year of high school, I am required to present a project that reflects my personal and technical growth throughtout this year. I believe this project fits the best for this task; compared to my previous projects, it aligns most closely with my future academic path. As I prepare to enter a five-year degree in Electronic Engineering, this design serves as a practical foundation for the challenges ahead.

Contents

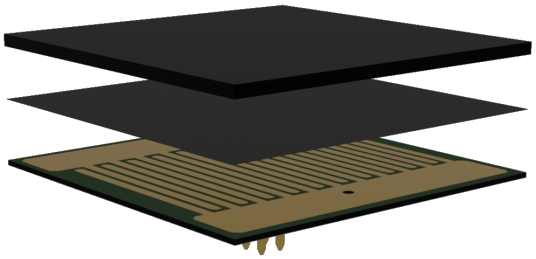
1 The Pad

Of course the first challenge was the pressure sensitive pads. I quickly realized that there is no commercial solution that really emulates the Akai pads–my research pivoted toward custom manufacturing. I then discovered **Velostat**, an electrically conductive material, generally used as a packaging material, which occurs to be **piezoresistive**, hence, its **resistance** changes with **pressure**. Thus, each pad would consist of a PCB with exposed traces and a square of velostat pressed against the surface, the latter will connect the two traces creating a **resistive path**. A simple **Wheatstone's bridge** will easily help us to calculate the resistance value.





Furthermore, I prioritized a **modular architecture**. By utilizing dedicated connectors, maintenance and upgrades become seamless, allowing for the effortless replacement of individual components without affecting the entire system. Finally, I integrated **LEDs** to provide intuitive and instantaneous **visual feedback**.



The final assembly includes an uppermost layer of **EVA foam**, which guarantees a sufficient downward pressure to maintain solid interface between the resistive material and the conductive traces.

2 The Pads Board

After getting the pads manufactured and shipped, I needed a dedicated interface **board** that would handle **eight buttons** in place. Since MCUs hardly support eight analog pins, letting a microcontroller handle individual analog value reads and LEDs is not the optimal solutions. Hence, the board integrates external ADCs communicating via the **I2C protocol**.
Adopting the same modular philosophy as the pads, the architecture simplifies the interface to just six connections: a single I2C line for the ADCs, separated (in order to split the load and decrease timing issues) from another I2C line for the GPIO extender, which will control LEDs, alongside two power pins.

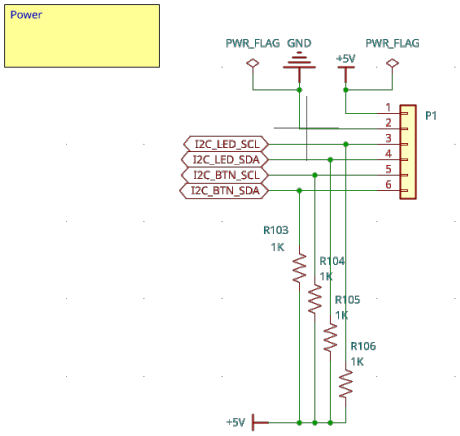


Figure 2: board powering

The **ADS1015** is a delta-sigma analog-to-digital converter that perfectly aligns with my project requirements. With **12-bit resolution**, four single-ended inputs and a sampling rate of up to **3300 SPS**, this component embodies all the qualities I was looking for–high convertion speed (one convertion takes approximately 0.3ms), a decent resolution (unsigned integers up to 4095) and its ability to handle four inputs via a single I2C address makes it a cost-effective and space-efficient solution for multi-channel sensing.
A Wheatstone's bridge-based approach outputs differential signals, which the ADS1015 does support; however, this reduces the capacity to only two inputs per converter. Scaling this to eight buttons would force us to use **four separate ADCs**, heavily increasing delays and causing a communication bottleneck, due to the fact that they all would need to share a single I2C line. To resolve this issue, the differential signal must be processed externally before reaching the ADC. While dedicated **instrumentation amplifiers** are ideal for this task, as they contain low-tolerance internal resistors, which obviously means a higher unit cost. Consequently, I figured out that emulating the instrumentation amplifier myself using three common **operational amplifiers** was a cheaper so-

lution, sacrificing an ultra-high sensing precision, which was not significant for this project.
More convenient was in fact the **TS914**, which packages **four rail-to-rail op-amps**, which let us keep tidiness on the board, furthermore isolating each pad, decreasing PCB complexity and maintaining a **clean layout**.

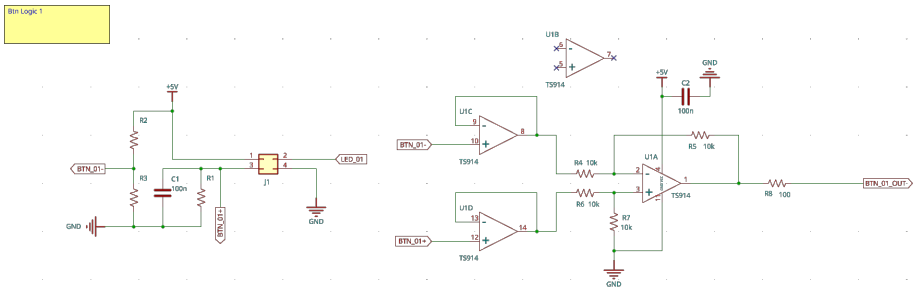


Figure 3: signal processing structure

Figure 3 represents the **complete signal processing chain** for a single pad; this architecture was subsequently replicated across all eight inputs.

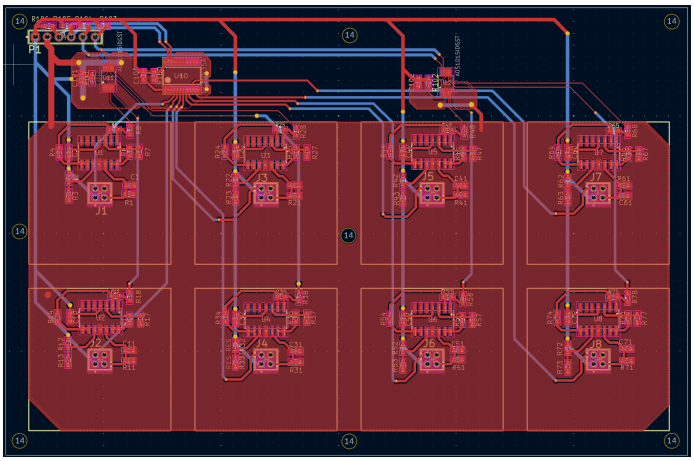


Figure 4: board PCB

As mentioned before, when designing the PCB, I focused on simplicity and maintaining a clean layout. Each pad is allocated a dedicated zone on the board, all the components are contained within that footprint, giving a clear visual representation of independency and isolation I was referring to.
Further information can be found inside the **GitHub repository**. All KiCad schematics and PCB layout fiels have been released as **open-source material**.

3 Development Tools

As already mentioned above, this board cannot work independently, it does not contain any processing unit that could elaborate the converted signals. Therefore this project will require another PCB to manage overall functionality of the sampler. At the time of this first draft of the paper, I still have yet to design such board, in which I plan to implement devices to allow **standalone functionalities**, so that no additional hardware (computers or MIDI synthesizers) is required for the instrument to work, like professional samplers.
Until the development of this final board, the only way to experiment with everything that was printed so far was to set up a breadboard environment, using an **Arduino Nano ESP32** I had at home. Having already some experience with *esp32 devices* and their development kit, I chose the **esp32s3** (the chip installed in the Arduino Nano ESP32) as a final choice for the board. Hence, the development started as a **C++ project** using the official espressif esp-idf framework.
Unintentionally, during this development period I was also interested in the **Rust programming language** and started reading the official book just for fun and out of curiosity. As a result, I became very fond of the language, and decided to give it a try and use it for a real project for the first time.
In the GitHub repository, I decided to keep both versions of the software (folders *esp32* and *esp32_rust*).
I opted for the esp-rs/esp-idf **hardware abstraction layer**, which **does** include Rust's **standard library** but is not carried on officially by espressif, but rather by the community. As a result, some C++ libraries might not have been ported to Rust yet, which makes it difficult to completely migrate from C++ to Rust. This was the case for **TinyUSB**, an important library I was relying on to declare a composite USB device recognisable by the computer.
To fix this issue, I chose to follow esp-idf template's way of **mixing** C++ and Rust, which suggests to keep the official's *idf.py tools* and C++, with the addition of a **Rust static library**. By using this method, I can still include esp-idf's C++ libraries and expose methods to the Rust library, covering for what Rust's unofficial framework has not implemented yet.

Thus, you can find a single C++ file that initializes USB capabilities using TinyUSB and exposes some of its functions to Rust.

```rust
// midi.rs

extern "C" {
    // C++ defined functions
    fn midi_mounted() -> bool;
    fn midi_out(packet: *const [u8; 4]);
}

pub struct MIDI;

// Wrapping unsafe methods
impl MIDI {
    pub fn send(packet: &[u8; 4]) {
        unsafe {
            if midi_mounted() {
                midi_out(packet);
            }
        }
    }
}
```

4 Pads Management

4.1 ADC Driver

As already mentioned, the pads board needs an MCU to configure the ADCs and read those resistance values. For starters, an **ADS1015 driver** needed to be developed. Module *ads1015/mod.rs* contains all the enums and methods to access and configure the ADCs via I2C communication.

```rust
pub struct ADS1015 {
    i2c: Arc<Mutex<I2cDriver<'static>>>,
    cfg_reg: AtomicU16,
    addr: u8,
}

impl ADS1015 {
    pub fn new(
        i2c: Arc<Mutex<I2cDriver<'static>>>,
        addr: u8
    ) -> Self {
        ADS1015 {
            i2c,
            cfg_reg: AtomicU16::new(0),
            addr,
        }
    }
    // ...
    pub fn read(&self) -> u16 {
        const REG_ADDR: [u8; 1] = [0x0];
        let mut data: [u8; 2] = [0, 0];

        {
            if let Ok(mut guard) = self.i2c.lock() {
                guard
                    .write_read(
                        self.addr, &REG_ADDR, &mut data, 1000
                    )
                    .unwrap();
            }
        }

        let raw_val = ((data[0] as u16) << 8) | (data[1] as u16);
        raw_val >> 4
    }
}
```

Recall that the two ADCs share a single I2C line, therefore, the *I2cDriver* also needs to be shared, which can be done safely by wrapping the struct in a *Mutex*. The ADS1015 structure was designed to work securely in **multi-threading environments**–the only mutable variable is marked as an *Atomic* and all the structure's methods work adopting the **interior mutability** pattern. Listing 4.1 shows how the *read* method fetches the latest converted value.

4.2 Pads Manager

Following with the **PadsManager** structure, responsible for the actual ADCs handling and reading. The instantiator creates two ADCs variables with the same configuration; the multiplexer is set to convert the first channel first, FSR (full-scale range) and data-rate are set to their maximum values, corresponding to $\pm 6.144V$ (which is clamped to 5V, the supply voltage) and 3300 SPS (samples per second). The operating mode is obviously set to **continuous**, while the comparator is disabled.

```
let i2c_config = I2cConfig::new().baudrate(Hertz(400_000));
let i2c_master = I2cDriver::new(i2c, sda, scl, &i2c_config)?;

let i2c_bus = Arc::new(Mutex::new(i2c_master));

let ads_cfg = ADS1015Config {
    mux_config: MuxConfig::MUX0,
    fsr_mode: Fsr6_144,
    op_mode: OpModeConfig::Continuous,
    data_rate: DataRateConfig::DataRate3300,
    comparator_mode: CompModeConfig::Traditional,
    comparator_polarity: CompPolarityConfig::ActiveLow,
    comparator_latching: CompLatchingConfig::NonLatching,
    queue_and_disable: CompQueueDisableConfig::DisableComp,
};

let ads1 = ADS1015::new(i2c_bus.clone(), addr1);
let ads2 = ADS1015::new(i2c_bus.clone(), addr2);

ads1.set_config(&ads_cfg)?;
ads2.set_config(&ads_cfg)?;
```

4.3 Sensing Task

PadsManager's main functionality is it's internal task.

```
/*
name: "pads_input_task",
stack_size: 4096,
priority: 15,
pin_to_core: Some(Core0),
*/

let mut channel: usize = 0;

loop {
    delay_us(750);

    let value1 = ads1.read();
    let value2 = ads2.read();

    if let Some(item) = process_pad_physics(channel as u8, &mut
    pads_settings[channel], value1) {
        queue.send_back(item, 0).unwrap();
    };

    if let Some(item) = process_pad_physics((channel as u8) + 4, &
    mut pads_settings[channel + 4], value2) {
        queue.send_back(item, 0).unwrap();
    };

    channel = (channel + 1) % 4;

    let cfg = MuxConfig::mux_from(channel as u8);
    ads1.set_mux(&cfg).unwrap();
    ads2.set_mux(&cfg).unwrap();
}
```

This is the heart of the pad sensing task (debug symbols were removed for simplicity). A task with relatively high priority (15) is pinned to the processor's Core0 and continuously fetches ADC values and sends events to another task using FreeRTOS's *Queue*.
Since a single conversion takes **0.3ms**, switching channels on the multiplexer needs some time, a **delay** is added to prevent incorrect data from previous conversions. Switching multiplexer channels and readings from both ADCs is done almost simultaneously to save time, eight pad measurements take around **1.2ms**. Moreover, software correction accounts for hardware inaccuracy and enhances the experience.

```rust
fn process_pad_physics(index: u8, settings: &mut DrumPad, value: u16) -> Option<PadInputEvent> {
    let now = timestamp();
    const SCAN_TIME_MS: u32 = 5;
    const RETRIGGER_MASK_MS: u32 = 30; // Helps to debounce by removing noise

    match settings.state {
        PadState::Idle => {
            if value > settings.threshold {
                settings.state = PadState::Attack;
                settings.peak = value;
                settings.timer_start = now;
            }
            None
        }
        PadState::Attack => {
            // Update peak if sample is higher
            if value > settings.peak {
                settings.peak = value;
            }

            // Wait 5ms
            if now - settings.timer_start >= SCAN_TIME_MS {
                settings.state = PadState::Sustain;

                let velocity = (settings.peak >> 5).min(127) as u8;

                Some(PadInputEvent {
                    index,
                    channel: settings.channel,
                    note: settings.note,
                    velocity: velocity,
                    event_type: PadInputEventType::MIDI(MidiType::NoteOn),
                })
            } else {
                None
            }
        }
        PadState::Sustain => {
            // Look for the value to drop below threshold
            const HYSTERESIS: f32 = 0.7;
            if value < (settings.threshold as f32 * HYSTERESIS) as u16 {
                settings.state = PadState::Release;
                settings.timer_start = now;

                Some(PadInputEvent {
                    index,
                    channel: settings.channel,
                    note: settings.note,
                    velocity: 0,
                    event_type: PadInputEventType::MIDI(MidiType::NoteOff),
                })
            } else {
```

```
                None
            }
        }
        PadState::Release => {
            if now - settings.timer_start >= RETRIGGER_MASK_MS {
                settings.state = PadState::Idle;
            }

            None
        }
    }
}
```

One pad press goes throught four different stages.

1. When the state is on **Idle**, the program waits for a value to go above a certain threshold, which can be configured individually for each pad.

2. From that moment, the program enters the **Attack** state–a **5ms window** is opened to look for the highest value, which will be used as the **pad velocity**, a **NoteOn** event is sent, consequently a MIDI event is transmitted to the USB host.

3. After that, the **Sustain** state starts, the function waits for a value to go below 70% of the configured threshold (allowing a 30% of hysteresis), a **NoteOff** event is sent, followed by a MIDI event.

4. Finally, the **Release** state waits 30ms for debouncing, preventing the pad to retrigger instantly.

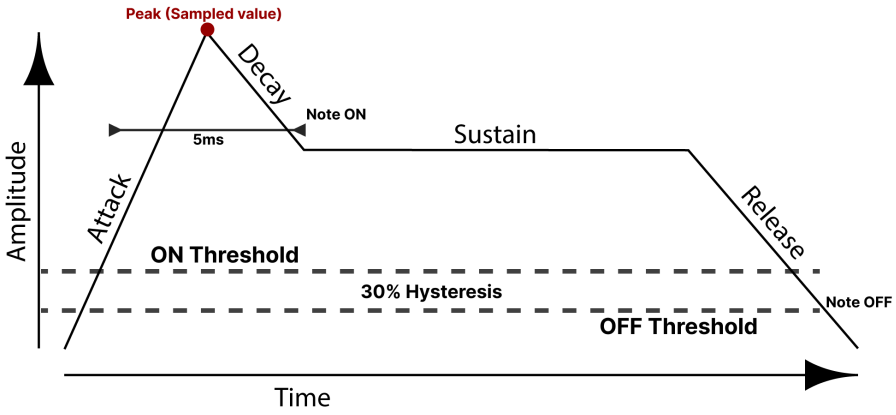


Figure 5: Software pads processing

4.4 Sending MIDI Events

Events that are sent to the **queue** are consumed by another task with a higher priority, which sends final MIDI events to the USB host, and also controls LEDs for the pads. Again, debug symbols were removed for simplicity.

```
/*
name: "pads_midi_task",
stack_size: 4096,
priority: 23,
pin_to_core: Some(Core::Core0),
*/

loop {
    if let Some((packet, _)) = queue.recv_front(delay::BLOCK) {
        match packet.event_type {
            PadInputEventType::MIDI(midi_type) => {
                let midi: (u8, u8) = midi_type.into();

                if !paused.load(Ordering::Relaxed) {
                    let bytes: [u8; 4] = [midi.0, midi.1 | (packet.
    channel & 0x0F), packet.note, packet.velocity];
                    MIDI::send(&bytes);
                }

                if let MidiType::NoteOn = midi_type {
                    leds_manager.set_led(packet.index, true);
                } else if let MidiType::NoteOff = midi_type {
                    leds_manager.set_led(packet.index, false);
                }
            }
            // ...
        }
    };
}
```

This code enables a 'free mode' for the drum pads, where pressing different pads sends configurable MIDI notes to the computer. By using a software sampler like **TX16Wx**, these MIDI events can be used to trigger and play back digital samples.

5 Quantizer

Currently, the system sends MIDI events as soon as a pad strike is sensed, ignoring the specific timing of the press. While this works for live performances, features like recording or sequencing require the program to quantize these events. By implementing a quantizer, the software can snap the timing of each note to a grid, ensuring the performance stays in sync with a set tempo. To achieve this, we define the smallest unit of time as a **pulse**. In a standard 4/4 time signature, a **loop** is divided into **four beats**, represented by the metronome clicks. Standard pulse resolutions typically range from **24 to 940 PPQ**, pulses (or ticks) per quarter note, where a 'quarter note' corresponds to what we earlier defined as a **beat**. For the development of the quantizer, it is practical to introduce an intermediate subdivision by dividing each beat (or quarter note) into **four steps** (1/16th notes).

Figure 6: The quantizer grid

In figure 6 the timing quantities defined by the bigger lines are the **beats**, each **beat** is then divided into PPQ amount of intervals, smaller lines are what we previously called **steps**.
To reach this level of precision, I'm relying on the ESP32's **general purpose timer**. We calculate the pulses interval as

$$TIMER\_STEP = \frac{60 \times TIMER\_RESOLUTION}{BPM \times PPQ} \tag{1}$$

Setting the timer resolution to a constant *TIMER_RESOLUTION*, the timer will call its interrupt exactly every pulse, based on the user-configurable BPM value and the PPQ resolution.
Note: all these values can be adjusted using the installed LCD display, we will cover graphics and screens later.

```
pub struct Quantizer {
    pub ticks: Arc<AtomicU8>,
    pub steps: Arc<AtomicU8>,
    timer: TimerDriver<'static>
}

impl Quantizer {
    pub fn new(notifier: Arc<Notifier>) -> Result<Self, EspError> {
        let ticks = Arc::new(AtomicU8::new(TICKS_PER_STEP - 1));
```

```
        let steps = Arc::new(AtomicU8::new(15));

        let mut timer_conf = TimerConfig::default();
        timer_conf.clock_source = ClockSource::Default;
        timer_conf.direction = CountDirection::Up;
        timer_conf.resolution = Hertz(TIMER_RESOLUTION);
        timer_conf.intr_priority = 3;

        let mut timer = TimerDriver::new(&timer_conf)?;
        timer.subscribe_default()?;
        timer.enable()?;

        // ... timer interrupt

        timer.start()?;

        Ok(Quantizer {
            ticks,
            steps,
            timer
        })
    }

    pub fn start(&self, bpm: u8) -> Result<(), EspError> {
        let time = 60u64 * TIMER_RESOLUTION as u64;
        let ticks = bpm as u64 * PPQ as u64;
        let timer_step = time / ticks;
        self.timer.set_alarm_action(Some(&AlarmConfig {
            alarm_count: timer_step,
            auto_reload_on_alarm: true,
            reload_count: 0,
            ..Default::default()
        }))?;

        Ok(())
    }
}
```

For simplicity, I decided to break down the timer counting into ticks (pulses) and steps, dividing a quarter note into four steps.

```
timer.subscribe(move |_| {
    let t = ticks.fetch_add(1, Ordering::SeqCst);

    if t + 1 >= TICKS_PER_STEP {
        ticks.store(0, Ordering::Relaxed);
        let s = (steps.load(Ordering::SeqCst) + 1) % 16;
        steps.store(s, Ordering::SeqCst);
    }

    notifier.notify_and_yield(NonZero::new(1).unwrap());
})?;
```

TICKS_PER_STEP is just PPQ divided by 4 .
Since we are in an interrupt context, the routine needs to be quick and non-blocking. An interrupt-safe *Notifier* structure is used to wake up another high-priority task.

```rust
/*
name: "quantizer_task",
stack_size: 4096,
priority: 24,
pin_to_core: Some(Core::Core1),
*/

let notification = Notification::new();
let quantizer = Arc::new(Quantizer::new(notification.notifier()).
    expect("Failed to create quantizer"));

const MIDI_SYNC_MSG: [u8; 4] = [0x0F, 0xF8, 0x00, 0x00];

loop {
    notification.wait_any();
    MIDI::send(&MIDI_SYNC_MSG);

    let ticks = quantizer.ticks.load(Ordering::SeqCst);
    let steps = quantizer.steps.load(Ordering::SeqCst);

    // notifying the quantizer
    if ticks == 0 {
        quantizer_seq_tx
            .send(SequencerMessage::NoteOn(steps))
            .ok();
    } else if ticks == 5 {
        quantizer_seq_tx
            .send(SequencerMessage::NoteOff(steps))
            .ok();
    }
}
```

A good practice when creating a MIDI device is to send a MIDI synchronization packet. This is a **System Real-Time message** used to synchronize all the connected equipment. It is a single byte status message (it does not contain any data) and it must be sent **24 times per quarter note**, hence it does not depend on our configured PPQ.
notification.wait_any() blocks the thread and waits for a notification from the interrupt, representing a pulse being triggered. After that, a MIDI sync message is transmitted and, as we will discuss later, a sequencer message is sent at the beginning (when ticks is equal to 0) and ending (last tick before a new step) of every step.

6 Sequencer

A sequencer enables the recording of a musical performance by capturing and storing note triggers relative to a master timeline. By mapping these events to specific timestamps, the sequencer can accurately reproduce the performance with consistent rhythmic integrity.
You can think of a sequencer performance as FL Studio's channel rack, each instrument has a different programmed pattern which repeats endlessly.

Figure 7: FL Studio's Channel Rack

Thus, a sequencer project is structured as a **list of tracks**.

```rust
pub struct SequencerTrack {
    pub loops: u8,
    pub resolution: SequencerResolution,
    pub note: u8,
    pub triggers: Vec<u8>,
}

pub struct SequencerProject {
    pub name: String,
    pub loops: u8,
    pub tracks: Vec<SequencerTrack>
}
```

While earlier we defined the largest unit of time as a loop, containing four quarter notes, a sequencer performance allows for more complex patterns with **multiple loops**. Each track has its own loops count, we do not need to program for four loops a simple hi-hat 2-step pattern, simply setting the *project_loop* to 4 and *track_loop* to 1 will do the trick. Obviously, *track_loop* needs to be a multiple of *project_loop* .
Notes can be programmed using the eight pads, when the Sampler is set to track programming mode, the pads LEDs stay on for set triggers, while project and tracks configurations can be edited from their screens. Since only eight pads for

a sequencer might not be ideal, I added a **resolution parameter** to each track. Resolutions range from each note representing a whole loop note, to represent a single quarter note.

```
pub enum SequencerResolution {
    Quarter     = 1,
    HalfBeat    = 2,
    Beat        = 4,
    Loop        = 16,
}
```

Mind that in this context, quarter notes and beats are **not** used as synonyms–I defined quarter notes as a quarter of a beat (1/16th of a loop).
The sequencer is handled by **another high-priority task**, which constantly waits for messages sent by the **quantizer** using Rust's *mpsc::channel()*.

```
/*
name: "sequencer_task",
stack_size: 4096,
priority: 23,
pin_to_core: Some(Core::Core1),
*/
loop {
    if let Ok(item) = sequencer_channel.recv() {
        if sequencer.enabled() {
            match item {
                SequencerMessage::NoteOn(step) => {
                    sequencer.step_trigger_on(step);
                }
                SequencerMessage::NoteOff(step) => {
                    sequencer.step_trigger_off(step);
                }
            }
        }
    }
}
```

From this moment, the sequencer handles the triggers.

```
pub fn step_trigger_on(&self, step: u8) {
    // ...
    for (track, state) in project.tracks.iter().zip(guard.iter_mut
    ()) {
        let track_step = step + (16 * (current_loop % track.loops))
    ;
        Self::handle_track_on(track, track_step, state);
    }
}
fn handle_track_on(track: &SequencerTrack, step: u8, state_trigger:
     &mut bool) {
    for trigger in track.triggers.iter() {
        let real_trigger = trigger * (track.resolution as u8);
        if real_trigger == step {
            *state_trigger = true;
            MIDI::send(&[0x09, 0x90, track.note, 127]);
        }
    }
}
```

```
17
18 pub fn step_trigger_off(&self, step: u8) {
19     // ...
20     for (track, state) in project.tracks.iter().zip(guard.iter_mut
       ()) {
21         let track_step = step + (16 * (current_loop % track.loops))
       ;
22         Self::handle_track_off(track, track_step, state);
23     }
24
25     if step == 15 {
26         self.current_loop.store((current_loop + 1) % project.loops,
        Ordering::Relaxed);
27     }
28 }
29 fn handle_track_off(track: &SequencerTrack, step: u8, state_trigger
       : &mut bool) {
30     for trigger in track.triggers.iter() {
31         let real_trigger = ((trigger + 1) * (track.resolution as u8
       )) - 1;
32         if real_trigger == step {
33             *state_trigger = false;
34             MIDI::send(&[0x08, 0x80, track.note, 0]);
35         }
36     }
37 }
```

Since track triggers are saved as pads indexes, before checking if a MIDI packet should be sent, the trigger value must be converted as a relative trigger.
(Image explanation)

7 Graphics

The most complex library I have written in this project is definitely the part that handles **graphics**. When designing such library, I aimed at some 'high' level abstraction which would separate hardware communication from UI composition. A lot of inspiration was taken from Android's (now) old style of structuring UIs and Applications–Android XML. At the moment, I am using a **text-based** 16x2 LCD display, thus, the graphics library heavily relies on Rust's *String* structure. The **GraphicsManager** is the core of this graphics library, this structure stores **screen instantiators**, installs **graphics drivers** and handles **graphics Events**. This graphics set of utilities relies on **graphics Events**, a screen is only refreshed when an update is sent. Events are sent from hardware controls, like pads pressing, controller movements etc.